*

C'est un schéma qui résume les *3 grands types de Machine Learning* 👇

1. Supervised Learning - Apprentissage supervisé

Le modèle apprend avec des données *étiquetées*. On lui donne l'entrée + la bonne réponse.

Il a 2 sous-catégories :

- *Classification* : prédire une catégorie

  Exemples :

  - *Fraud detection* = Détection de fraude
  - *Email Spam Detection* = Détection de spams
  - *Diagnostics* = Diagnostic médical
  - *Image Classification* = Classification d'images

- *Regression* : prédire une valeur numérique

  Exemples :

  - *Risk Assessment* = Évaluation des risques
  - *Score Prediction* = Prédiction de score

2. Unsupervised Learning - Apprentissage non supervisé

Le modèle apprend avec des données *non étiquetées*. Il doit trouver les patterns tout seul.

Il a 2 sous-catégories :

- *Dimensionality Reduction* = Réduction de dimension

  Exemples :

  - *Text Mining* = Fouille de texte
  - *Face Recognition* = Reconnaissance faciale
  - *Big Data Visualization* = Visualisation de big data
  - *Image Recognition* = Reconnaissance d'images

- *Clustering* = Regroupement

  Exemples :

  - *Biology* = Biologie
  - *City Planning* = Urbanisme

- *Targeted Marketing* = Marketing ciblé

3. Reinforcement Learning - Apprentissage par renforcement

Le modèle apprend par essais/erreurs avec des récompenses/pénalités.

Exemples :

- *Gaming* = Jeux vidéo

- *Finance Sector* = Secteur financier

- *Manufacturing* = Industrie

- *Inventory Management* = Gestion des stocks

- *Robot Navigation* = Navigation de robots

Tu veux que je t'explique plus en détail un des 3 types ? Par exemple la différence entre Classification et Regression ?

Apprentissage non supervisé ☺*

Type d'apprentissage automatique où le modèle est

entraîné sur des données non étiquetées pour découvrir

des structures ou des patterns cachés.

L'apprentissage supervisé est un type d'apprentissage automatique où le modèle

est entraîné sur des données étiquetées

Données étiquetées = بيانات مُعَلَّمَة (bayanat mu'allaama)

= données avec des exemples déjà classés (ex : emails spam/non spam)Données non étiquetées = بيانات غيْر مُعَلَّمَة (bayanat ghayr mu'allaama)

= données brutes, sans catégorie

Descente de Gradient # ##########################

Algorithme d'optimisation utilisé pour minimiser une

fonction de coût

- *r polynomial* : Modèle pour modéliser une relation non linéaire entre X et Y en utilisant des termes polynomiaux.

● **Overfitting:** Un modèle qui est trop complexe et qui mémorise les données

d'entraînement au lieu de jeneralise

● **Underfitting:** Un modèle qui est trop simple et qui ne capture pas les

complexités des données

- svm :Algorithme d apprentisage supervisée qui cherche à trouver l'hyperplan optimal séparant les classes.

☺ *Arbre de décision* : Modèle qui prend des décisions en divisant les données en branches selon des règle sur les attributs, jusqu'à une prédiction finale (feuille).

KNN classe un point selon ses K voisins les plus proches. Vote majoritairree pour décider la classe. ☺s ☺

Un Fondement de l'Apprentissage Supervisé,Utilisée pour établir une relation linéaire entre une variable dépendante (que l'on cherche à prédire) et une ou plusieurs variables indépendantes (les prédicteurs).

En d'autres termes, elle cherche à trouver la droite (ou l'hyperplan dans un espace à plus de deux dimensions) qui ajuste au mieux les données.

- **Par renforcement:** Le modèle apprend en interagissant avec un environnement. Il reçoit des récompenses ou des pénalités en fonction de ses actions et ajuste sa stratégie pour maximiser les récompenses.

```python
import numpy as np

def descente_gradient(X, y, alpha=0.01, iterations=1000):
    m = len(y)
    theta = np.zeros(X.shape[1])
    couts = []

    for _ in range(iterations):
        predictions = X.dot(theta)
        erreurs = predictions - y
        gradient = (1/m) * X.T.dot(erreurs)
        theta -= alpha * gradient
        couts.append((1/(2*m)) * np.sum(erreurs**2))
```

```python
import numpy as np  # Importer la bibliothèque NumPy

def descente_gradient(X, y, learning_rate=0.01, iterations=1000):
    """
    Implémentation de la descente de gradient pour la régression linéaire.

    Paramètres :
    - X : matrice des caractéristiques
    - y : vecteur cible
    - learning_rate : taux d'apprentissage (défaut : 0.01)
    - iterations : nombre d'itérations (défaut : 1000)
    """
    m = len(y)  # Nombre d'échantillons
    theta = np.zeros(X.shape[1])  # Initialiser les paramètres à zéro
    couts = []  # Liste pour stocker les coûts à chaque itération

    for _ in range(iterations):  # Boucle sur le nombre d'itérations
        predictions = X.dot(theta)  # Prédictions itératives
        erreurs = predictions - y  # Erreurs entre prédictions et cibles
        gradient = (1/m) * X.T.dot(erreurs)  # Calcul du gradient
```

```
        theta -= learning_rate * gradient  # Mise à jour des paramètres
        couts.append(np.mean(erreurs**2) / 2)  # Calcul et stockage du coût

    return theta, couts  # Retourner les paramètres optimisés et les coûts
```

- *Définition* :

- ***Équation* : Y = β0 + β1X + ε**
  - Y : variable dépendante
  - X : variable indépendante
  - β0 : intercept (ordonnée à l'origine)
  - β1 : pente (coefficient de X)

  - ε : erreur
- *Objectif* : Trouver β0 et β1 qui minimisent l'erreur (somme des carrés des résidus).

الانحدار الخطي البسيط : نموذج للتنبؤ بمتغير مستمر بناءً على متغير مستقل واحد

Régression Linéaire Multiple ☺

- ***Équation* : Y = β0 + β1X1 + β2X2 + ... + βnXn + ε**
  - Y : variable dépendante
  - X1, X2...Xn : variables indépendantes
  - β0 : intercept
  - β1, β2...βn : coefficients
  - ε : erreur
- *Objectif* : Trouver les coefficients (β) qui minimisent l'erreur (somme des carrés des résidus).

Régression Polynomiale ☺

- *Équation* **: Y = β0 + β1X + β2X² + ... + βnX^n + ε**
  - Y : variable dépendante
  - X : variable indépendante
  - β0, β1...βn : coefficients
  - n : degré du polynôme
  - ε : erreur
- *Objectif* : Trouver les coefficients qui minimisent l'erreur.
- *Risque* : surajustement (overfitting) si degré trop élevé.
- *Utilisation* : modéliser des relations non linéaires.

Exemple : Y = β0 + β1X + β2X² ☺

Algorithmes ☺

Régression Logistique ☺

La régression logistique est une méthode statistique utilisée pour modéliser la relation entre une variable dépendante binaire (ayant deux valeurs possibles, par exemple 0 ou 1) et un ensemble de variables indépendantes.

- *Équation* : p = 1 / (1 + exp(-z)), où z = β0 + β1X1 + β2X2 + ... + βnXn

  - p : probabilité de la classe positive (1)

  - X1, X2...Xn : variables indépendantes

  - β0, β1...βn : coefficients

- *Fonction de coût* : log-vraisemblance (log-loss)

- *Objectif* : Trouver les coefficients qui maximis ent la log-vraisemblance (ou minimisent le log-loss).

- *Algorithme* : Descente de gradient, Newton-Raphson...

- *Utilisation* : classification binaire, probabilités de classe.

- *Évaluation* : accuracy, précision, rappel, F1-score, AUC-ROC...

En arabe ☺

الانحدار اللوجستي : نموذج للتنبؤ بمتغير ثنائي بناءً على متغيرات مستقلة.

Avantages ☺

- Interprétable

- Efficace pour données linéairement séparables

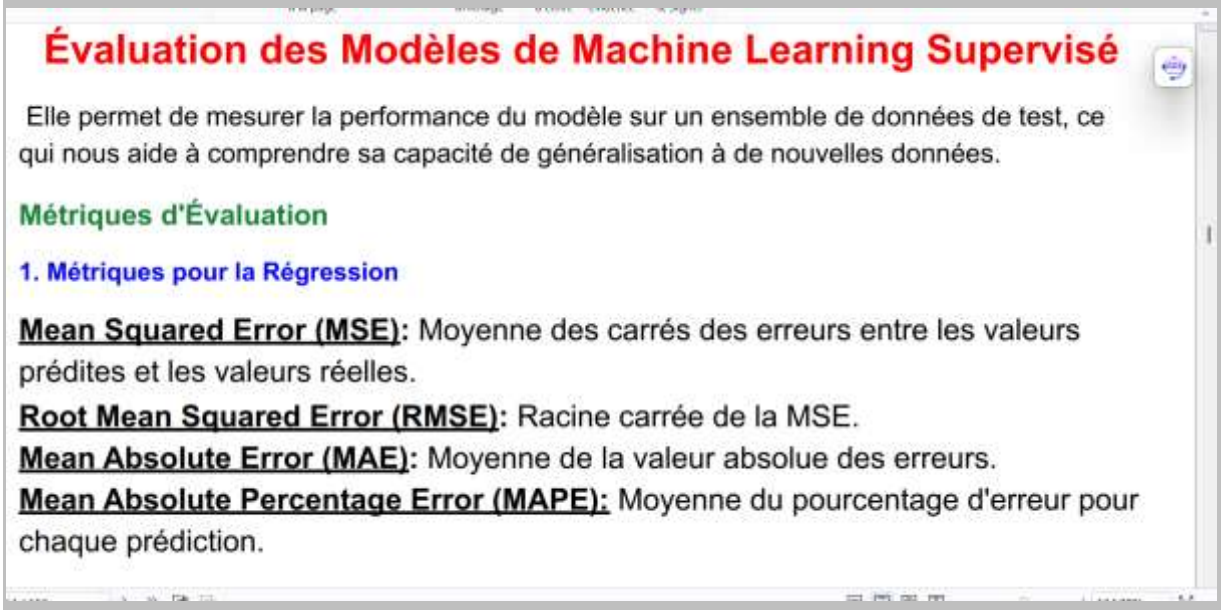
- Probabilités de classe

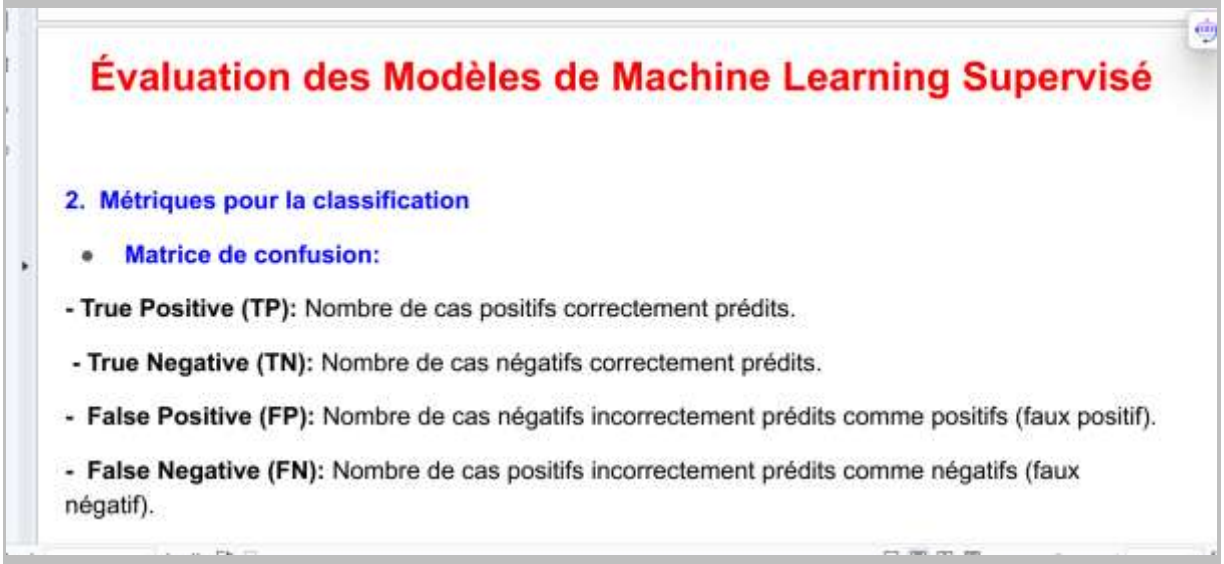
Inconvénients ☺

- Suppose linéarité

- Sensible aux outliers

Métriques d'Évaluation 😊





1. *MSE (Erreur Quadratique Moyenne)*

   - **Formule : $MSE = (1/n) * \Sigma(y_i - \hat{y}_i)^2$**

   - Utilisation : régression (linéaire, polynomiale...)

2. *RMSE (Erreur Quadratique Moyenne Racine)*

- **Formule : RMSE = √MSE**

- Utilisation : régression

3. *MAE (Erreur Absolue Moyenne)*

- **Formule : MAE = (1/n) * Σ|y_i - ŷ_i|**

- Utilisation : régression

4. *R^2 (Coefficient de Détermination)*

- **Formule : $R^2 = 1 - (\Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2)$**

- Utilisation : régression

5. *Accuracy (Précision Globale)*

- **Formule : Accuracy = (TP + TN) / (TP + TN + FP + FN)**

- Utilisation : classification

6. *Précision*

- **Formule : Précision = TP / (TP + FP)**

- Utilisation : classification

7. *Rappel*

- **Formule : Rappel = TP / (TP + FN)**

- Utilisation : classification

8. *F1-score*

- **Formule : F1 = 2 * (Précision * Rappel) / (Précision + Rappel)**

- Utilisation : classification

9. *Log-loss (Perte Logarithmique)*

- **Formule : Log-loss = -(1/n) * Σ[y_i * log(ŷ_i) + (1-y_i) * log(1-ŷ_i)]**

- Utilisation : classification (régression logistique)

Légende ☺

- TP : vrais positifs

- TN : vrais négatifs

- FP : faux positifs

- FN : faux négatifs

- y_i : valeur réelle

- ŷ_i : valeur prédite

- ȳ : moyenne des y_i

*SVM (Support Vector Machine)

Définition
- Algorithme de classification supervisée qui cherche à trouver l'hyperplan optimal séparant les classes.

Formules Mathématiques
1. *Hyperplan*
   - Équation : w^T x + b = 0
   f(x) = wT * x + b
   - w : vecteur normal à l'hyperplan
   - b : biais
   - x : point dans l'espace

Hyperplan: *C'est* le plan qui sépare au mieux les données en différentes classes.

Vecteurs de support :Les points de données ou les vecteurs les plus proches de l'hyperplan

● Marge: C'est la distance entre l'hyperplan et les points de données les plus proches,

appelés vecteurs de support.

● Optimisation: L'algorithme SVM cherche à trouver l'hyperplan qui maximise cette marge.

Plus la marge est grande, meilleure est la généralisation du modèle

Le Noyau (Kernel)

Le noyau est un concept fondamental dans les Machines à Vecteurs de Support (SVM). Il joue un rôle crucial en permettant aux SVM de gérer des problèmes de classification non linéaires.

Types de noyaux couramment utilisés :

- Noyau linéaire: Le plus simple, il correspond à un produit scalaire standard. Il est utilisé lorsque les données sont linéairement séparables.
- Noyau polynomial: Permet de capturer des relations polynomiales entre les données.
- Noyau RBF (Radial Basis Function): Très populaire, il permet de capturer des relations non linéaires complexes.
- Autres noyaux: *Sigmoïde, etc.*

Choix du noyau :

Le choix du noyau dépend du problème à résoudre et des caractéristiques des données. Il n'existe pas de règle universelle. On utilise souvent une approche empirique en testant différents noyaux et en sélectionnant celui qui donne les meilleurs résultats

Types de SVM

SVM peut être de deux types :

- *SVM linéaire*: est utilisé pour les données séparables linéairement, ce qui signifie que si un ensemble de données peut être classé en deux classes *en utilisant une seule ligne droite*, ces données sont alors appelées données séparables linéairement
- SVM non linéaire : *est utilisé pour les données non séparables linéairement, ce qui signifie que si un ensemble de*

données ne peut pas être classé à l'aide d'une ligne droite,

- Donc, pour séparer ces points de données non linéaires, nous devons ajouter une dimension supplémentaire z.
- Il peut être calculé comme suit :

$z = x_2 + y_2$

-

Les noyaux SVMsont des équations mathématiques complexes
qui effectuent des transformations complexes de données pour
optimiser l'hyperplan

Voici la fonction qui définit le noyau linéaire :

$f(x) = wT * x + b$

Voici la fonction pour un noyau polynomial :

$f(x1, x2) = (a + x1T * x2)b$

Voici l'équation pour un noyau RBF

$f(x1, x2) = \exp(-gamma * ||x1-x2||2)$

*Avantages

Domaine

Détection de visage:
Classification des images
Catégorisation de texte et d'hypertexte
Détection de Spam

🌳 Cours complet : Arbres de Décision (Machine Learning)

---

1. 📌 Définition

Un **arbre de décision** est un algorithme de **Machine Learning supervisé** utilisé pour :

- ✓ Classification (ex : spam / non spam)
- ✓ Régression (ex : prédire un prix)

Il prend des décisions sous forme de **questions successives**.

---

2. □ Principe

On découpe les données avec des questions :

☞ Exemple :

- Est-ce que Age < 30 ?
- Est-ce que Salaire > 5000 ?

Le but :
➞ Obtenir des groupes **les plus purs possibles**

3. 🌿 Structure

- **Racine** : début de l'arbre
- **Nœud** : condition
- **Branche** : résultat
- **Feuille** : décision finale

4. ⚙ Construction de l'arbre

Étapes :

1. Calculer un critère (Entropie ou Gini)
2. Choisir la meilleure variable
3. Diviser les données
4. Répéter (récursif)
5. Arrêter si :
    - données pures
    - profondeur max atteinte

5. 📏 Critères mathématiques

◆ 5.1 Entropie (mesure du désordre)

$H(S) = -\sum_{i=1}^{n} p_i \log_2(p_i)$

✓ Interprétation :

- H = 0 → données pures
- H élevé → mélange

◈ Exemple simple :

Dataset :

- 2 Oui
- 2 Non

☞ p(Oui) = 0.5, p(Non) = 0.5

$H(S) = - (0.5 \log_2 0.5 + 0.5 \log_2 0.5)$
$H(S) = 1$ → **désordre maximal**

◆ 5.2 Gain d'information

$Gain(S,A)=H(S)−\sum v∈Values(A)|Sv||S|H(Sv)Gain(S, A) = H(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} H(S_v)Gain(S,A)=H(S)−∑v∈Values(A)|S||Sv|H(Sv)$

✓ Idée :

➞ Combien on réduit le désordre après division

✓ Règle :

☞ On choisit la variable avec **le plus grand gain**

◆ 5.3 Indice de Gini

$Gini=1−\sum i=1npi2Gini = 1 - \sum_{i=1}^{n} p_i^2Gini=1−∑i=1npi2$

✓ Interprétation :

- Gini = 0 → parfait
- Gini élevé → mélange

◈ Exemple :

- p(Oui) = 0.5
- p(Non) = 0.5

$Gini = 1 - (0.5^2 + 0.5^2)$
$Gini = 0.5$

---

6. 🌲 Types d’arbres

✓ Classification

- Sortie : classe
- Exemple : malade / sain

✓ Régression

- Sortie : valeur
- Exemple : prix maison

---

7. ⚠ Overfitting

● Problème :

Arbre trop complexe → mémorise les données

✓ Solutions :

- max_depth
- min_samples_split
- pruning (élagage)

---

8. ✂ Pruning (élagage)

- Supprimer des branches inutiles
- Simplifier le modèle
- Améliorer la généralisation

---

9. 👍 Avantages

✓ Facile à comprendre
✓ Visualisable
✓ Pas besoin de normalisation
✓ Gère les relations non linéaires
